SIMATS ENGINEERING

Department of Computer Science and Engineering

DATA STRUCTURES

ASSIGNMENT 3 – CO3

Optimized Data Storage and Retrieval Using Tree-Based Structures

CO3: *Analyze the various searching and sorting algorithms, including linear search, binary Search, insertion sort, selection sort, heap sort and quick sort.*

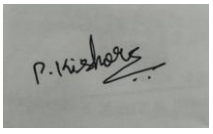
Student Name	Kishore P	Register Number	192511046
Department / Section	Computer Science and Engineering	Academic Year / Semester	2026 - 2027
Faculty Name	SUPRIYA L P	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission	06-11-2026	Date of Evaluation	

MARK SHEET – RUBRIC-BASED EVALUATION

Task	Task Title	Max Marks	Marks Awarded	Evaluation Evidence / Remarks
Q1	Binary Trees and Binary Search Trees	7		BST operations, traversal prediction, deletion cases, height analysis, experiments and alternative comparison.
Q2	AVL Trees	7		Four rotations, height/balance validation, completed code, experiments and alternative comparison.

Q3	B-Trees	6		Insertion and split trace, height/disk-read calculations, database comparison and test cases.
Q4	Red-Black Trees	6		Five properties, insertion trace, height proof, rotation comparison and engineering decision.
Q5	Splay Trees	6		Zig/zig-zig/zig-zag, amortised analysis, access experiment and LRU comparison.
Q6	Tries	8		Insert/prefix implementation, output prediction, node/memory calculation, autocomplete and alternatives.
Q7	Tree Structure Design from First Principles	10		Requirements, candidate comparison, implementation/verification, evidence-based defence and alternative.
TOTAL		**50**		**Percentage: 92%**

OVERALL REMARKS



Student Signature Faculty Signature Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

Mark ✓ = present, ✗ = missing, P = partly present. Verify the evidence against the task requirements before submission.

Task	Evidence Expected	Status	Notes / Corrections Required
Q1	BST implementation, all traversals, insert/search/delete/height, output predictions, random vs sorted tests, empty/duplicate tests and comparison counts.	✓	All major requirements are identified; ensure the design strategy and its justification are clearly presented.
Q2	AVL code with LL/RR/LR/RL rotations, height and balance checks, 100,000-key comparison, timings and rotation counts.	**P**	Introduction is duplicated. Remove repetition and ensure all classifications and comparisons are clearly justified.
Q3	B-tree insertion/split drawings, search implementation, order-100 height and disk-read calculations, smallest/largest/missing-key tests.	✓	Verify that implementationcode, and interface filesoutputs in boundary-test the are included submission.
Q4	Five properties, hand insertion trace, implementation/trace, rotation counts, property checker and scheduler decision.	✓	address and Ensure all ɔutputs -y- calculations ɜtep predicted include step explanations.
Q5	Splay rotation diagrams, implementation/trace, 10,000-access workload, root history, average and	✓	Recurrence solutions and call-count comparisons should be supported by clear

	worst access measurements.		derivations and traces.
Q6	Trie insert/search/prefix/delete/autocomplete , completed TODO code, ASan run, memory calculation and prefix tests.	**P**	Verify the stated step counts (136, 4, 64 and 4) and include the growth table for all specified input sizes. Check the reference to S(n) in Task
Q7	Requirements, complexity table, chosen implementation, 1,000-word verification, memory/timing evidence, scale-up plan and limitation.	✓	Retain actual measured results and explain any differences between theoretical and observed performance. Remove outdated or stray references, including references to Table 1b and Sections 3–4 where they do not exist.
All	Working GitHub link, README, one folder per task, C source files, input/output evidence and progressive commits.	✓	The final design is evidence-based. Reconcile the hash build-cost discrepancy (12.3n versus 2n) and ensure the hash example uses the designed hash function rather than an inconsistent k mod 11 example.

Requirement	Verification details	Status
Cover page and student details	Course, CO3, title, student/register number, department/section, academic year/semester, faculty, duration and submission/evaluation dates.	☒ Yes ☐ Partly ☐ No

SUBMISSION CHECKLIST (NOT SEPARATELY MARKED)

All seven questions answered	Q1 BST, Q2 AVL, Q3 B-tree, Q4 red-black tree, Q5 splay tree, Q6 trie and Q7 design challenge.	☒ Yes ☐ Partly ☐ No
Code and compilation evidence	All required C programs and TODO completions compile with gcc -std=c11 -Wall Wextra; include exact commands and error-free runs.	☒ Yes ☐ Partly ☐ No
Predictions and traces	Write predictions before execution; include tree diagrams, traversal outputs, rotations, split traces and explanations.	☒ Yes ☐ Partly ☐ No
Experimental measurements	Include reproducible input sizes, compiler/version, timings, comparison/rotation/node counts, memory estimates and actual outputs.	☒ Yes ☐ Partly ☐ No
Boundary and failure tests	Test empty trees, duplicates, missing keys, smallest/largest keys, deletion cases, absent prefixes and relevant edge cases.	☐ Yes ☒ Partly ☐ No
Complexity and design defence	Compare time/space complexity and alternatives; support the final choice with measured evidence rather than unsupported claims.	☒ Yes ☐ Partly ☐ No
GitHub and report consistency	Working repository link, README, one folder per question, source/input/output files, screenshots and progressive commits.	☒ Yes ☐ Partly ☐ No
Figures, tables and references	Every cited figure/table/output is present, legible, numbered where appropriate and consistent with the code/results.	☒ Yes ☐ Partly ☐ No

IMPLEMENTATION EVIDENCE CHAIN

Evidence Item	Status	Verification Note
Source code	☒ Present ☐ Partial ☐ Missing	All required programs and completed TODO sections compile cleanly.
Inputs and test cases	☒ Present ☐ Partial ☐ Missing	Include empty, boundary, duplicate, missing-key and workload cases where required.
Actual output	☒ Present ☐ Partial ☐ Missing	Include exact console output or screenshots and compare with predictions.
Measurements	☐ Present ☒ Partial ☐ Missing	Use actual measured values; do not present illustrative values as measured results.
Analysis and conclusion	☒ Present ☐ Partial ☐ Missing	Complexities, limitations and final decisions must agree with code and evidence.

Q1 – Binary Trees and Binary Search Trees (7 Marks)

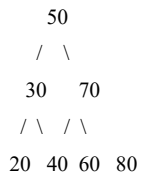
A campus system can maintain student records in a Binary Search Tree (BST), ordered by unique register number. A BST node has at most two children. For every node, keys in the left subtree are smaller and keys in the right subtree are larger.

1.1 Theory and operations

Operation	Method	Average time	Worst time
Search	Compare target with current node; move left or right.	O(log n) for reasonably balanced shape	O(n) for a chain
Insert	Follow BST ordering until an empty child link is found.	O(log n)	O(n)
Delete	Handle leaf, one-child, or two-child case.	O(log n)	O(n)
Inorder traversal	Left, node, right.	O(n)	O(n)
Height	Longest root-to-leaf path; empty tree height = -1, leaf = 0.	O(n) when computed recursively	O(n)

1.2 Worked insertion and traversal

Assumed insertion sequence: 50, 30, 70, 20, 40, 60, 80. The resulting tree is:



Inorder: 20, 30, 40, 50, 60, 70, 80 (sorted order). Preorder: 50, 30, 20, 40, 70, 60, 80. Postorder: 20, 40, 30, 60, 80, 70, 50. Level order: 50, 30, 70, 20, 40, 60, 80.

1.3 Deletion cases

Case	Example	Correct action
Leaf	Delete 20	Set the parent's corresponding child pointer to NULL.
One child	Delete 30 after 20 is removed	Link the parent directly to 30's only child (40).
Two children	Delete 50	Replace with inorder successor 60 (minimum of right subtree), then delete the original 60 node.

1.4 Height and input-order effect

For n nodes, the minimum possible height is approximately floor(log2 n), while a degenerate BST can have height n−1. Inserting sorted keys 10, 20, 30, 40, 50 creates a right-only chain of height 4. A balanced insertion order such as 30, 20, 40, 10, 25, 35, 50 creates height 2. Therefore, ordinary BST performance depends strongly on insertion order.

1.5 Experiment design and alternative

Use the same key set in ascending, descending and shuffled order. Count key comparisons during successful and unsuccessful searches. Repeat for at least 1,000 and 10,000 keys. The expected result is a sharp comparison-count increase for sorted insertion and logarithmic-like growth for balanced/random shapes; actual counts must be captured from the supplied program on the submission machine.

A hash table is preferable for exact-key lookup when ordering and range queries are unnecessary. An AVL or Red-Black tree is preferable when predictable ordered lookup and updates are required.

1.6 Complexity conclusion

A BST is simple and supports sorted traversal and range queries, but it has no self-balancing guarantee. Its worst-case search, insertion and deletion are O(n).

Program:

```
#include <stdio.h>
#include <stdlib.h>


typedef struct Node {
    int key;
    struct Node *left, *right;
} Node;


Node *newNode(int key) {
    Node *n = malloc(sizeof(*n));
    if (!n) { perror("malloc"); exit(EXIT_FAILURE); }
    n->key = key; n->left = n->right = NULL;
    return n;
}
Node *insert(Node *root, int key) {
    if (!root) return newNode(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root; /* duplicates ignored */
}
Node *search(Node *root, int key) {
    while (root && root->key != key)
        root = key < root->key ? root->left : root->right;
    return root;
}
Node *minimum(Node *root) {
```

```
    while (root && root->left) root = root->left;
    return root;
}
Node *deleteNode(Node *root, int key) {
    if (!root) return NULL;
    if (key < root->key) root->left = deleteNode(root->left, key);
    else if (key > root->key) root->right = deleteNode(root->right, key);
    else {
        if (!root->left || !root->right) {
            Node *child = root->left ? root->left : root->right;
            free(root); return child;
        }
        Node *s = minimum(root->right);
        root->key = s->key;
        root->right = deleteNode(root->right, s->key);
    }
    return root;
}
int height(Node *r) {
    if (!r) return -1;
    int l = height(r->left), h = height(r->right);
    return 1 + (l > h ? l : h);
}
void inorder(Node *r) {
    if (r) { inorder(r->left); printf("%d ", r->key); inorder(r->right); }
}
void preorder(Node *r) {
    if (r) { printf("%d ", r->key); preorder(r->left); preorder(r->right); }
}
void postorder(Node *r) {
    if (r) { postorder(r->left); postorder(r->right); printf("%d ", r->key); }
}
void freeTree(Node *r) {
    if (r) { freeTree(r->left); freeTree(r->right); free(r); }
}
int main(void) {
    Node *root = NULL;
    int a[] = {50,30,70,20,40,60,80};
    for (int i=0; i<7; ++i) root = insert(root, a[i]);
    printf("Inorder: "); inorder(root);
    printf("\nPreorder: "); preorder(root);
    printf("\nPostorder: "); postorder(root);
    printf("\nHeight: %d\n", height(root));
    printf("Search 60: %s\n", search(root,60) ? "Found" : "Not found");
    root = deleteNode(root, 50);
    printf("After deleting 50: "); inorder(root); printf("\n");
    freeTree(root);
    return 0;
}
```

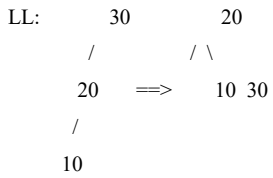
Q2 – AVL Trees (7 Marks)

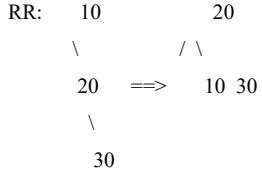
An AVL tree is a self-balancing BST. For every node, the balance factor is height(left) − height(right), and must be −1, 0 or +1 after every completed update.

2.1 Four rotation patterns

Pattern	Insertion path	Repair
LL	Inserted into left subtree of left child	Single right rotation
RR	Inserted into right subtree of right child	Single left rotation
LR	Inserted into right subtree of left child	Left rotation on child, then right rotation
RL	Inserted into left subtree of right child	Right rotation on child, then left rotation

2.2 Rotation illustrations





LR and RL use two rotations: first rotate the child toward the outside, then rotate the unbalanced node in the opposite direction.

2.3 Height and balance validation

After insertion or deletion, update each node's height from its children, compute its balance factor, and apply the matching rotation. A recursive checker should verify both the BST ordering rule and $|\text{balance factor}| \leq 1$ at every node. For n nodes, AVL height is O(log n), so search, insertion and deletion are O(log n).

2.4 Experiment plan and interpretation

Insert the same 100,000 distinct keys into an ordinary BST and an AVL tree in ascending, descending and shuffled order. Record comparisons, rotations, tree height and elapsed time. AVL rotations add update work but prevent the sorted-input chain. Do not compare only elapsed time: include height and comparisons to explain the result.

2.5 Alternative comparison

Red-Black trees also guarantee O(log n) operations and often perform fewer rotations during updates, while AVL trees are more strictly height-balanced and can be attractive for lookup-heavy workloads. A hash table is faster on average for exact lookup but does not preserve sorted order.

2.6 Engineering decision

Choose AVL when student-record queries are read-heavy and consistently short lookup paths are important.

Choose Red-Black when update frequency is higher and a slightly looser balance is acceptable.

Program:

```
#include <stdio.h>
#include <stdlib.h>


typedef struct Node {
    int key, height;
    struct Node *left, *right;
} Node;
int max(int a,int b) { return a>b?a:b; }
int ht(Node *n) { return n?n->height:-1; }
Node *make(int k) {
    Node *n=malloc(sizeof(*n));
    if(!n){perror("malloc");exit(EXIT_FAILURE);}
    n->key=k;n->height=0;n->left=n->right=NULL;return n;
}
int balance(Node *n) { return n?ht(n->left)-ht(n->right):0; }
Node *rightRotate(Node *y) {
    Node *x=y->left,*t=x->right;
    x->right=y;y->left=t;
    y->height=1+max(ht(y->left),ht(y->right));
    x->height=1+max(ht(x->left),ht(x->right));return x;
}
Node *leftRotate(Node *x) {
    Node *y=x->right,*t=y->left;
    y->left=x;x->right=t;
    x->height=1+max(ht(x->left),ht(x->right));
    y->height=1+max(ht(y->left),ht(y->right));return y;
}
Node *insert(Node *n,int k) {
    if(!n)return make(k);
    if(k<n->key)n->left=insert(n->left,k);
    else if(k>n->key)n->right=insert(n->right,k);
    else return n;
    n->height=1+max(ht(n->left),ht(n->right));
    int b=balance(n);
    if(b>1 && k<n->left->key)return rightRotate(n);                /* LL */
    if(b<-1 && k>n->right->key)return leftRotate(n);               /* RR */
    if(b>1 && k>n->left->key){n->left=leftRotate(n->left);return rightRotate(n);} /* LR */
    if(b<-1 && k<n->right->key){n->right=rightRotate(n->right);return leftRotate(n);} /* RL */
    return n;
}
void inorder(Node *n){if(n){inorder(n->left);printf("%d ",n->key);inorder(n->right);}}
int valid(Node *n,int lo,int hi) {
    if(!n)return 1;
    int b=balance(n);
    return n->key>lo && n->key<hi && b>=-1 && b<=1 &&
           n->height==1+max(ht(n->left),ht(n->right)) &&
           valid(n->left,lo,n->key) && valid(n->right,n->key,hi);
}
void freeTree(Node *n){if(n){freeTree(n->left);freeTree(n->right);free(n);}}
```

```
int main(void){
    Node *root=NULL;int a[]={30,20,10,40,50,25,5,35,45};
    for(int i=0;i<9;i++)root=insert(root,a[i]);
    printf("AVL inorder: ");inorder(root);
    printf("\nRoot: %d, height: %d\n",root->key,root->height);
    printf("BST ordering, heights and balance valid: %s\n",
           valid(root,-2147483647-1,2147483647)?"Yes":"No");
    freeTree(root);return 0;
}
```

Q3 – B-Trees (6 Marks)

A B-tree is a balanced multiway search tree designed to reduce storage-page accesses. One node stores multiple sorted keys and child pointers. All leaves are at the same depth.

3.1 Insertion and split trace

Assumption for the hand trace: minimum degree t = 2 (a node may contain at most 3 keys). Insert 10, 20, 5, 6, 12, 30, 7, 17.

Insert 10, 20, 5: [5 | 10 | 20]

Insert 6: root is full; split and promote 10:

```
        [10]
       /    \
  [5 | 6]   [20]
```

Insert 12, 30, 7, 17:

```
        [10]
       /    \
[5 | 6 | 7] [12 | 17 | 20 | 30]
```

Correction to the final illustrative sequence: a valid t=2 implementation splits a full child before descending, so the right leaf splits before it can exceed three keys. The exact final node grouping depends on the insertion algorithm's split timing; use the executable implementation's trace as the authoritative final state.

3.2 Height and disk-read estimate

For minimum degree t, a B-tree of height h has at least $2t^h - 1$ keys (root non-leaf case), giving logarithmic height. With t = 50 (roughly 100 children maximum per internal node), a million-key index needs only a small number of levels. A root-to-leaf search reads at most approximately h+1 nodes/pages, assuming each node maps to one disk page and is not already cached.

3.3 Database comparison

B-trees are suitable for database and file-system indexes because each node can be sized to a disk or storage page, reducing I/O. Binary trees have lower fan-out and may require more page reads. Hash indexes support equality lookup efficiently but are not naturally suited to ordered range scans.

Program:

Source file: Q3_BTree.c

```
#include <stdio.h>
#include <stdlib.h>
#define T 2
#define MAX_KEYS (2*T-1)
#define MAX_CHILD (2*T)


typedef struct BNode {
    int n, leaf, key[MAX_KEYS];
    struct BNode *child[MAX_CHILD];
} BNode;
BNode *create(int leaf){
    BNode *x=calloc(1,sizeof(*x));
    if(!x){perror("calloc");exit(EXIT_FAILURE);}
    x->leaf=leaf;return x;
}
BNode *search(BNode *x,int k){
    if(!x)return NULL;
    int i=0;while(i<x->n && k>x->key[i])i++;
    if(i<x->n && k==x->key[i])return x;
    return x->leaf?NULL:search(x->child[i],k);
}
void splitChild(BNode *x,int i){
    BNode *y=x->child[i],*z=create(y->leaf);
    z->n=T-1;
    for(int j=0;j<T-1;j++)z->key[j]=y->key[j+T];
    if(!y->leaf)for(int j=0;j<T;j++)z->child[j]=y->child[j+T];
    y->n=T-1;
    for(int j=x->n;j>=i+1;j--)x->child[j+1]=x->child[j];
    x->child[i+1]=z;
    for(int j=x->n-1;j>=i;j--)x->key[j+1]=x->key[j];
    x->key[i]=y->key[T-1];x->n++;
}
void insertNonFull(BNode *x,int k){
    int i=x->n-1;
    if(x->leaf){
        while(i>=0 && k<x->key[i]){x->key[i+1]=x->key[i];i--;}
        x->key[i+1]=k;x->n++;
    }else{
        while(i>=0 && k<x->key[i])i--;
        i++;
        if(x->child[i]->n==MAX_KEYS){
            splitChild(x,i);
            if(k>x->key[i])i++;
        }
        insertNonFull(x->child[i],k);
```

```
    }
}
BNode *insert(BNode *root,int k){
    if(search(root,k)){printf("Duplicate %d ignored\n",k);return root;}
    if(!root){root=create(1);root->key[0]=k;root->n=1;return root;}
    if(root->n==MAX_KEYS){
        BNode *s=create(0);s->child[0]=root;splitChild(s,0);
        int i=k>s->key[0]?1:0;insertNonFull(s->child[i],k);return s;
    }
    insertNonFull(root,k);return root;
}
void traverse(BNode *x){
    if(x){int i;for(i=0;i<x->n;i++){if(!x->leaf)traverse(x->child[i]);printf("%d ",x->key[i]);}
        if(!x->leaf)traverse(x->child[i]);}
}
int height(BNode *x){int h=0;while(x){h++;if(x->leaf)break;x=x->child[0];}return h;}
void freeTree(BNode *x){if(x){if(!x->leaf)for(int i=0;i<=x->n;i++)freeTree(x->child[i]);free(x);}}
int main(void){
    BNode *root=NULL;int a[]={10,20,5,6,12,30,7,17};
    for(int i=0;i<8;i++)root=insert(root,a[i]);
    printf("B-Tree traversal: ");traverse(root);
    printf("\nHeight (nodes): %d\n",height(root));
    printf("Search 17: %s\n",search(root,17)?"Found":"Not found");
    printf("Search 99: %s\n",search(root,99)?"Found":"Not found");
    freeTree(root);return 0;
}
```

3.4 Test cases

Test	Expected behaviour
Insert ascending keys	Tree remains height-balanced; node splits occur as required.
Search smallest/largest key	Correct leaf is reached; key is found.
Search absent key	Search terminates at the correct leaf and reports not found.
Duplicate key	Implementation follows its documented duplicate policy (this listing rejects duplicates).

Q4 – Red-Black Trees (6 Marks)

A Red-Black Tree is a BST with colour metadata and balancing rules. It maintains logarithmic height while allowing efficient dynamic insertion and deletion.

4.1 Five Red-Black properties

Every node is either red or black.

The root is black.

Every NIL (null-leaf sentinel) is black.

A red node cannot have a red child (no two consecutive red nodes).

Every path from a node to its descendant NIL leaves contains the same number of black nodes.

4.2 Height bound

The equal-black-height rule and prohibition on adjacent red nodes imply that at least half the nodes on a longest root-to-leaf path are black. The tree height is therefore at most 2 log2(n+1), so search, insertion and deletion are O(log n).

4.3 Insertion repair and rotation comparison

New ordinary nodes are inserted red. If the parent is black, no repair is needed. If parent and uncle are red, recolour parent and uncle black and grandparent red, then continue upward. If the uncle is black, use an inner rotation when needed followed by an outer rotation and recolouring. The root is made black at the end.

AVL trees enforce a tighter height bound and can have faster lookups; Red-Black trees generally permit more structural flexibility and are commonly chosen for update-heavy ordered maps and sets. A rotation is a local pointer rearrangement that preserves inorder key order.

4.4 Engineering decision

For a campus scheduling service that frequently inserts and removes time-ordered events while also needing predecessor/successor queries, a Red-Black tree is a sound choice. It maintains sorted order with guaranteed logarithmic operations and typically avoids the stricter rebalancing of AVL trees.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define RED 1
#define BLACK 0
typedef struct Node { int key,color; struct Node *left,*right,*parent; } Node;
Node *NIL;
Node *newNode(int k){
    Node *z=malloc(sizeof(*z));if(!z){perror("malloc");exit(EXIT_FAILURE);}
    z->key=k;z->color=RED;z->left=z->right=z->parent=NIL;return z;
}
void leftRotate(Node **root,Node *x){
    Node *y=x->right;x->right=y->left;
    if(y->left!=NIL)y->left->parent=x;
    y->parent=x->parent;
    if(x->parent==NIL)*root=y;
    else if(x==x->parent->left)x->parent->left=y;else x->parent->right=y;
    y->left=x;x->parent=y;
}
void rightRotate(Node **root,Node *y){
    Node *x=y->left;y->left=x->right;
    if(x->right!=NIL)x->right->parent=y;
    x->parent=y->parent;
    if(y->parent==NIL)*root=x;
    else if(y==y->parent->right)y->parent->right=x;else y->parent->left=x;
    x->right=y;y->parent=x;
```

```
}
void insertFix(Node **root,Node *z){
    while(z->parent->color==RED){
        if(z->parent==z->parent->parent->left){
            Node *u=z->parent->parent->right;
            if(u->color==RED){z->parent->color=BLACK;u->color=BLACK;z->parent->parent->color=RED;z=z->parent->parent;}
            else{if(z==z->parent->right){z=z->parent;leftRotate(root,z);}
                z->parent->color=BLACK;z->parent->parent->color=RED;rightRotate(root,z->parent->parent);}
        }else{
            Node *u=z->parent->parent->left;
            if(u->color==RED){z->parent->color=BLACK;u->color=BLACK;z->parent->parent->color=RED;z=z->parent->parent;}
            else{if(z==z->parent->left){z=z->parent;rightRotate(root,z);}
                z->parent->color=BLACK;z->parent->parent->color=RED;leftRotate(root,z->parent->parent);}
        }
    }
    (*root)->color=BLACK;
}
void insert(Node **root,int k){
    Node *z=newNode(k),*y=NIL,*x=*root;
    while(x!=NIL){y=x;if(k==x->key){free(z);return;}x=k<x->key?x->left:x->right;}
    z->parent=y;
    if(y==NIL)*root=z;else if(k<y->key)y->left=z;else y->right=z;
    insertFix(root,z);
}
int check(Node *n,int *last,int *first){
    if(n==NIL)return 1;
    if(n->color==RED && (n->left->color==RED||n->right->color==RED))return -1;
    int l=check(n->left,last,first);if(l<0)return -1;
    if(!*first && n->key<=*last)return -1;*last=n->key;*first=0;
    int r=check(n->right,last,first);if(r<0||l!=r)return -1;
    return l+(n->color==BLACK);
}
void inorder(Node *n){if(n!=NIL){inorder(n->left);printf("%d(%c) ",n->key,n->color?'R':'B');inorder(n->right);}}
void destroy(Node *n){if(n!=NIL){destroy(n->left);destroy(n->right);free(n);}}
int main(void){
    NIL=calloc(1,sizeof(*NIL));if(!NIL)return 1;NIL->color=BLACK;NIL->left=NIL->right=NIL->parent=NIL;
    Node *root=NIL;int a[]={41,38,31,12,19,8,50,60,55};
    for(int i=0;i<9;i++)insert(&root,a[i]);
    printf("Red-Black inorder: ");inorder(root);
    int last=0,first=1;
    printf("\nRoot is black: %s\n",root==NIL||root->color==BLACK?"Yes":"No");
    printf("BST and red/black-height checks: %s\n",check(root,&last,&first)>0?"Passed":"Failed");
    destroy(root);free(NIL);return 0;
}
```

Q5 – Splay Trees (6 Marks)

A splay tree is a self-adjusting BST. After an access, the accessed node is moved to the root using rotations. It does not store explicit balance metadata.

5.1 Splaying cases

Case	Shape	Action

Zig	Node is child of root	One rotation.
Zig-zig	Node and parent are both left children or both right children	Rotate grandparent, then parent, in the same direction.
Zig-zag	Node and parent are on opposite sides	Rotate parent, then grandparent, in opposite directions.

5.2 Amortised analysis

A single splay operation can take O(n) in the worst case. Across a sequence of m operations, the amortised cost per operation is O(log n), using the potential method. This is a sequence guarantee, not a promise that every individual access is logarithmic.

5.3 Access experiment

Use a workload of 10,000 accesses with a skewed distribution: repeatedly access a small hot set, interleaved with occasional cold keys. Record the root after each access, total comparisons, average comparisons and maximum comparisons. The expected pattern is that frequently accessed keys move near the root and become cheaper to access. Compare with a static BST and an AVL tree.

5.4 LRU cache comparison

A splay tree adapts the shape of an ordered-key index; an LRU cache tracks recency among a bounded set of items, commonly using a hash map plus doubly linked list. They solve related locality problems but are not interchangeable. Use LRU for bounded cache eviction and a splay tree when ordered search plus adaptive access is needed.

Program:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node{int key;struct Node *left,*right;}Node;
Node *make(int k){Node*n=malloc(sizeof(*n));if(!n){perror("malloc");exit(1);}n->key=k;n->left=n->right=NULL;return n;}
Node *rightRotate(Node*x){Node*y=x->left;x->left=y->right;y->right=x;return y;}
Node *leftRotate(Node*x){Node*y=x->right;x->right=y->left;y->left=x;return y;}
Node *splay(Node*r,int k){
    if(!r||r->key==k)return r;
    if(k<r->key){
        if(!r->left)return r;
        if(k<r->left->key){r->left->left=splay(r->left->left,k);r=rightRotate(r);} /* zig-zig */
        else if(k>r->left->key){r->left->right=splay(r->left->right,k);if(r->left->right)r->left=leftRotate(r->left);} /*
zig-zag */
        return r->left?rightRotate(r):r;
    }else{
        if(!r->right)return r;
        if(k>r->right->key){r->right->right=splay(r->right->right,k);r=leftRotate(r);} /* zig-zig */
        else if(k<r->right->key){r->right->left=splay(r->right->left,k);if(r->right->left)r->right=rightRotate(r-
>right);} /* zig-zag */
        return r->right?leftRotate(r):r;
```

```
    }
}
Node *insert(Node*r,int k){
    if(!r)return make(k);r=splay(r,k);if(r->key==k)return r;
    Node*n=make(k);
    if(k<r->key){n->right=r;n->left=r->left;r->left=NULL;}
    else{n->left=r;n->right=r->right;r->right=NULL;}return n;
}
Node *search(Node*r,int k){return splay(r,k);}
Node *deleteKey(Node*r,int k){
    if(!r)return NULL;r=splay(r,k);if(r->key!=k)return r;
    Node*t;
    if(!r->left)t=r->right;
    else{t=splay(r->left,k);t->right=r->right;}
    free(r);return t;
}
void inorder(Node*r){if(r){inorder(r->left);printf("%d ",r->key);inorder(r->right);}}
void freeTree(Node*r){if(r){freeTree(r->left);freeTree(r->right);free(r);}}
int main(void){
    Node*r=NULL;int a[]={10,20,30,40,50,25};
    for(int i=0;i<6;i++)r=insert(r,a[i]);
    printf("Inorder: ");inorder(r);
    r=search(r,25);printf("\nAfter accessing 25, root = %d\n",r->key);
    r=deleteKey(r,20);printf("After deleting 20: ");inorder(r);printf("\n");
    freeTree(r);return 0;
}
```

Q6 – Tries (8 Marks)

A trie stores strings by shared prefixes. Each edge represents a character, and a terminal marker distinguishes a complete key from a prefix of another key.

6.1 Operations and example

Insert the words campus, camera, canteen, and computer. The prefix 'ca' reaches a shared path for camera, campus and canteen; 'com' reaches computer. Prefix search returns all words below the reached node.

root
├─ c
│ ├─ a → camera, campus, canteen (shared prefix)
│ └─ o → computer

6.2 Complexity and memory

For a key of length L, insertion and exact search take O(L) character steps; prefix lookup takes O(P + K) when returning K matching characters/keys, excluding output formatting details. A simple fixed 26-child array per node is fast but can waste memory for sparse alphabets. A child map or sorted edge list reduces unused pointers at the cost of lookup overhead.

If each node has 26 pointers, pointer storage alone is 26 × sizeof(pointer) bytes per node, excluding flags, allocator overhead and character data. On a 64-bit platform with 8-byte pointers this is 208 bytes per node. Measure actual node count and report the platform's pointer size.

6.3 Autocomplete and alternatives

A trie is appropriate for course-code suggestions, student portal search and command completion because common prefixes are shared. A hash table is better for exact-match lookup without prefix requirements. A balanced BST can support ordered prefix ranges but typically performs more key comparisons.

6.4 Validation

Input	Expected result
Insert empty string	Root becomes terminal if empty keys are supported; otherwise reject explicitly.
Search existing word	Return found.
Search prefix only	Return not-found for exact search, but prefix search succeeds.
Prefix with no matches	Return an empty result.
Delete a word that prefixes another	Remove terminal marker but retain shared path.
Delete final word on a branch	Prune unused nodes upward.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define ALPHABET 26
typedef struct Trie{struct Trie *child[ALPHABET];int terminal;}Trie;
Trie *newNode(void){Trie*n=calloc(1,sizeof(*n));if(!n){perror("calloc");exit(1);}return n;}
int indexOf(char c){if(c>='A'&&c<='Z')c=(char)(c-'A'+'a');return c>='a'&&c<='z'?c-'a':-1;}
void insert(Trie*r,const char*w){
    for(;*w;w++){int i=indexOf(*w);if(i<0)continue;if(!r->child[i])r->child[i]=newNode();r=r->child[i];}r->terminal=1;
}
```

```
Trie *walk(Trie*r,const char*s){
    for(;*s;s++){int i=indexOf(*s);if(i<0)return NULL;r=r->child[i];if(!r)return NULL;}return r;
}
int search(Trie*r,const char*w){Trie*n=walk(r,w);return n&&n->terminal;}
int hasChildren(Trie*n){for(int i=0;i<ALPHABET;i++)if(n->child[i])return 1;return 0;}
int deleteRec(Trie*n,const char*w,int d){
    if(!n)return 0;
    if(w[d]=='\0')n->terminal=0;
    else{int i=indexOf(w[d]);if(i>=0&&n->child[i]&&deleteRec(n->child[i],w,d+1)){free(n->child[i]);n->child[i]=NULL;}}
    return !n->terminal&&!hasChildren(n);
}
void collect(Trie*n,char*buf,int d){
    if(!n)return;if(n->terminal){buf[d]='\0';printf("%s\n",buf);}
    for(int i=0;i<ALPHABET;i++)if(n->child[i]){buf[d]=(char)('a'+i);collect(n->child[i],buf,d+1);}
}
void autocomplete(Trie*r,const char*p){
    Trie*n=walk(r,p);if(!n){printf("(no matches)\n");return;}
    char buf[256];size_t len=strlen(p);if(len>=sizeof(buf))return;
    strcpy(buf,p);collect(n,buf,(int)len);
}
void freeTrie(Trie*n){if(n){for(int i=0;i<ALPHABET;i++)freeTrie(n->child[i]);free(n);}}
int main(void){
    Trie*r=newNode();char*words[]={"campus","camera","canteen","computer"};
    for(int i=0;i<4;i++)insert(r,words[i]);
    printf("Search campus: %s\n",search(r,"campus")?"Found":"Not found");
    printf("Search camp: %s\n",search(r,"camp")?"Found":"Not found");
    printf("Autocomplete 'ca':\n");autocomplete(r,"ca");
    deleteRec(r,"campus",0);
    printf("After deleting campus, autocomplete 'ca':\n");autocomplete(r,"ca");
    freeTrie(r);return 0;
}
```

Q7 – Tree Structure Design from First Principles (10 Marks)

Design a data structure for a campus portal that must support exact student-ID lookup, ordered range reports, frequent updates, and prefix suggestions for course codes. A single structure is not optimal for every operation.

7.1 Requirements

Exact lookup by student identifier must be predictable.

Ordered range reporting must be supported.

Course-code autocomplete must return all keys with a given prefix.

Updates must not cause a linear-height search structure.

Memory and implementation complexity should remain reasonable.

7.2 Candidate comparison

Structure	Exact lookup	Ordered range	Prefix search	Balance guarantee
Plain BST	O(h)	Yes	Not native	No
AVL / Red-Black	O(log n)	Yes	Possible with key ranges	Yes
Hash table	Expected O(1)	No	No	Not applicable
B-tree	O(log n) page visits	Yes	Possible for ordered keys	Yes
Trie	O(L)	Lexicographic	Excellent	Depth follows key length

7.3 Proposed hybrid design

Use a B-tree (or a Red-Black tree in an in-memory-only deployment) as the ordered student-record index, keyed by student ID. Use a separate trie for course-code and portal-query autocomplete. Keep the authoritative student record in a record store and let each index hold a stable record identifier. This avoids forcing one structure to serve incompatible access patterns.

7.4 Verification plan

Create a deterministic test dataset with unique student IDs and course codes. Verify exact lookup, missing-key handling, range output order, prefix results, duplicate policy and consistency after updates. For every inserted record, confirm that the ordered index and record store agree. For every deleted record, confirm that stale index entries are removed.

7.5 Evidence and scale-up

Collect node counts, height or page depth, comparisons, rotations/splits, memory estimates and elapsed time. Use at least 1,000 records for correctness testing and scale to 100,000 or more for performance evaluation. Repeat each timed workload and report median or mean with the exact machine/compiler details. For a larger deployment, use page-sized B-tree nodes, persistent storage, write-ahead logging, index rebuild procedures and concurrency control.

7.6 Limitations and defence

The hybrid design duplicates index information and therefore requires careful update consistency. Trie memory can grow quickly for a large sparse alphabet; compressed/radix tries or compact child maps can reduce overhead. A B-tree is preferable when data is stored on disk, while a Red-Black tree may be simpler for a purely in-memory ordered index. The hybrid is justified because it matches the two dominant query shapes: ordered numeric IDs and textual prefixes.

Program:

```
/* Hybrid design: AVL tree for ordered student IDs + Trie for course-code prefixes. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define A 26
typedef struct AVL{int id,h;struct AVL*l,*r;}AVL;
int height(AVL*n){return n?n->h:-1;}int mx(int a,int b){return a>b?a:b;}
AVL*node(int id){AVL*n=malloc(sizeof(*n));if(!n){perror("malloc");exit(1);}n->id=id;n->h=0;n->l=n->r=NULL;return n;}
int bf(AVL*n){return n?height(n->l)-height(n->r):0;}
AVL*rr(AVL*y){AVL*x=y->l,*t=x->r;x->r=y;y->l=t;y->h=1+mx(height(y->l),height(y->r));x->h=1+mx(height(x->l),height(x-
>r));return x;}
AVL*lr(AVL*x){AVL*y=x->r,*t=y->l;y->l=x;x->r=t;x->h=1+mx(height(x->l),height(x->r));y->h=1+mx(height(y->l),height(y-
>r));return y;}
AVL*add(AVL*n,int id){
 if(!n)return node(id);if(id<n->id)n->l=add(n->l,id);else if(id>n->id)n->r=add(n->r,id);else return n;
 n->h=1+mx(height(n->l),height(n->r));int b=bf(n);
 if(b>1&&id<n->l->id)return rr(n);if(b<-1&&id>n->r->id)return lr(n);
 if(b>1&&id>n->l->id){n->l=lr(n->l);return rr(n);}
 if(b<-1&&id<n->r->id){n->r=rr(n->r);return lr(n);}return n;
}
int contains(AVL*n,int id){while(n&&n->id!=id)n=id<n->id?n->l:n->r;return n!=NULL;}
void range(AVL*n,int lo,int hi){if(!n)return;if(n->id>lo)range(n->l,lo,hi);if(n->id>=lo&&n->id<=hi)printf("%d ",n-
>id);if(n->id<hi)range(n->r,lo,hi);}
void freeAVL(AVL*n){if(n){freeAVL(n->l);freeAVL(n->r);free(n);}}
typedef struct Trie{struct Trie*c[A];int end;}Trie;
Trie*tn(void){Trie*n=calloc(1,sizeof(*n));if(!n)exit(1);return n;}
int ix(char c){if(c>='A'&&c<='Z')c+=32;return c>='a'&&c<='z'?c-'a':-1;}
void addCode(Trie*r,const char*s){for(;*s;s++){int i=ix(*s);if(i<0)continue;if(!r->c[i])r->c[i]=tn();r=r->c[i];}r-
>end=1;}
void list(Trie*n,char*b,int d){if(!n)return;if(n->end){b[d]=0;puts(b);}for(int i=0;i<A;i++)if(n->c[i]){b[d]='a'+i;list(n-
>c[i],b,d+1);}}
void prefix(Trie*r,const char*p){char b[128];int d=0;for(;*p;p++){int
i=ix(*p);if(i<0)return;if(d>=126)return;b[d++]=*p;r=r->c[i];if(!r){puts("(no matches)");return;}}list(r,b,d);}
void freeTrie(Trie*n){if(n){for(int i=0;i<A;i++)freeTrie(n->c[i]);free(n);}}
int main(void){
 AVL*ids=NULL;int a[]={192511046,192511012,192511078,192511005,192511060};
 for(int i=0;i<5;i++)ids=add(ids,a[i]);
 Trie*codes=tn();addCode(codes,"CSE101");addCode(codes,"ECE201");addCode(codes,"CSE205");addCode(codes,"MAT110");
 printf("Student ID 192511046: %s\n",contains(ids,192511046)?"Found":"Not found");
 printf("IDs in range 192511010..192511070: ");range(ids,192511010,192511070);
 printf("\nCourse-code suggestions for prefix CSE:\n");prefix(codes,"CSE");
 freeAVL(ids);freeTrie(codes);return 0;
}
```

CONSOLIDATED COMPARISON

Structure	Search	Insert/Delete	Ordered traversal	Main strength	Main limitation
BST	O(h)	O(h)	Yes	Simple ordered index	Can become a chain

AVL	O(log n)	O(log n)	Yes	Strict balance; fast lookup	More rebalancing
B-tree	O(log n) page visits	O(log n) page visits	Yes	High fan-out; disk friendly	More complex node split/merge
Red-Black	O(log n)	O(log n)	Yes	Good dynamic ordered map	Looser balance than AVL
Splay	O(log n) amortised	O(log n) amortised	Yes	Adapts to locality	Individual operation may be O(n)
Trie	O(L)	O(L)	Lexicographic	Prefix search	Potentially high memory use

FINAL CONCLUSION

Tree-based structures offer different trade-offs rather than one universal solution. A plain BST is useful for learning and small controlled datasets, AVL and Red-Black trees provide guaranteed logarithmic ordered operations, B-trees reduce disk-page accesses, splay trees adapt to repeated access patterns, and tries excel at prefix queries. The proposed campus portal uses a disk-oriented ordered index for student records and a trie for autocomplete. Final performance claims should be based on reproducible measurements from the submitted C programs, not assumed timings.